

SimPT: An Automated Simulation Framework for Intervention-Based VLA Post-Training

Anonymous Authors

Abstract—Human-in-the-loop (HIL) post-training improves vision-language-action (VLA) policies through expert corrections, but costly human supervision and repeated real-robot evaluation limit algorithm exploration and development. We introduce SimPT (Simulation-based Post-Training), a simulation framework that automates intervention-based VLA post-training by integrating policy rollouts, corrective data collection, policy training, and standalone evaluation. To automatically determine intervention timing and recovery strategies, we develop an Agent-Compiled Progress Graph that tracks task progress using privileged simulator state and retains stable snapshots during policy execution. After a rollout fails, a CuRobo-based expert attempts recovery from these snapshots, starting at more advanced task stages and falling back to earlier ones as needed to generate corrective trajectories. We implement and compare four post-training methods, RECAP, HG-DAGger, DPO+SFT, and FlowPRO, across seven RoboTwin manipulation tasks. Relative to a fixed-interval snapshot baseline, the Progress Graph reduces recovery attempts by 75.0% and increases recovery success from 93.4% to 98.9%. Using SimPT, we develop RECAP-CF-LPF, a refinement of RECAP that combines cross-fitting and low-pass filtering of value predictions. This refinement increases the overall success rate from 76.50% to 79.36% in held-out simulation trials. In real-robot HIL post-training, it increases the success rate from 72% to 96% across 25 scenarios of an Ethernet cable-insertion task. These results demonstrate the utility of SimPT for exploring and developing VLA post-training algorithms.

I. INTRODUCTION

Generalist vision-language-action (VLA) policies map visual observations and language instructions directly to robot actions [1], [2]. When these policies encounter states outside the demonstration distribution, or out-of-distribution (OOD) states, they may produce inappropriate actions [3]. Such errors can drive the robot further from the demonstration distribution and compound over time [4]. Human-in-the-loop (HIL) robot learning addresses this problem through expert takeover, using corrective demonstrations to update the policy [5], [6], [7]. However, developing HIL algorithms on physical robots requires substantial human effort to collect corrections and repeated hardware use for evaluation. Environmental variation further complicates reproducible comparisons [8]. These costs and sources of variability limit systematic exploration and development of HIL post-training algorithms.

Simulation offers controllable scenes, repeatable trials, and scalable data collection for developing and evaluating HIL algorithms before hardware experiments [9], [10]. Platforms such as RoboTwin [11] and SIMPLER [8] support demonstration generation, policy evaluation, and sim-to-real research, while automated HIL post-training remains comparatively

underexplored. IntervenGen extends simulation-based data generation to corrective behavior by expanding a small set of human interventions into synthetic recovery trajectories, but relies on human experts to determine when and how to intervene in the initial recovery demonstrations [12]. Automating these decisions in simulation requires selecting suitable recovery states along a failed policy rollout and determining how an expert should complete the task from those states. Both depend on task progress. Vision-language models (VLMs) can monitor execution, but repeated visual queries incur inference overhead [13]. CuRobo-based scripted experts can execute predefined task sequences from initial states [14], but the motion planner itself does not infer task progress or select corrective actions for states reached by the policy.

Motivated by these challenges, we introduce *SimPT*, a simulation framework that integrates policy rollouts, automated corrective data collection, policy training, and standalone evaluation to support the exploration and development of VLA post-training algorithms (Fig. 1). To guide recovery, an agent compiles task information into an *Agent-Compiled Progress Graph*. During policy execution, the graph tracks task progress through fixed checks on privileged simulator state and retains stable snapshots without further agent queries. After rollout failure, SimPT attempts expert recovery from snapshots at more advanced task stages and falls back to earlier ones as needed. A CuRobo-based expert resumes execution from each restored state, and successful recovery trajectories are retained for policy post-training.

Using SimPT, we implement and compare HG-DAGger [6], RECAP [15], DPO+SFT [16], and FlowPRO [17] on seven RoboTwin manipulation tasks. In independent data collections, the Progress Graph requires 75.0% fewer recovery attempts than a fixed-interval snapshot baseline. We further use SimPT to develop RECAP-CF-LPF, a refinement of RECAP that combines cross-fitting (CF) and Butterworth low-pass filtering (LPF) of value predictions. This combined refinement increases the overall success rate from 76.50% to 79.36% in held-out simulation trials. Applied to real-robot HIL post-training, it increases success from 72% to 96% across 25 scenarios of an Ethernet cable-insertion task. Our contributions are threefold:

- We introduce SimPT, a simulation framework that automates the intervention-based VLA post-training workflow to support systematic algorithm exploration and development.

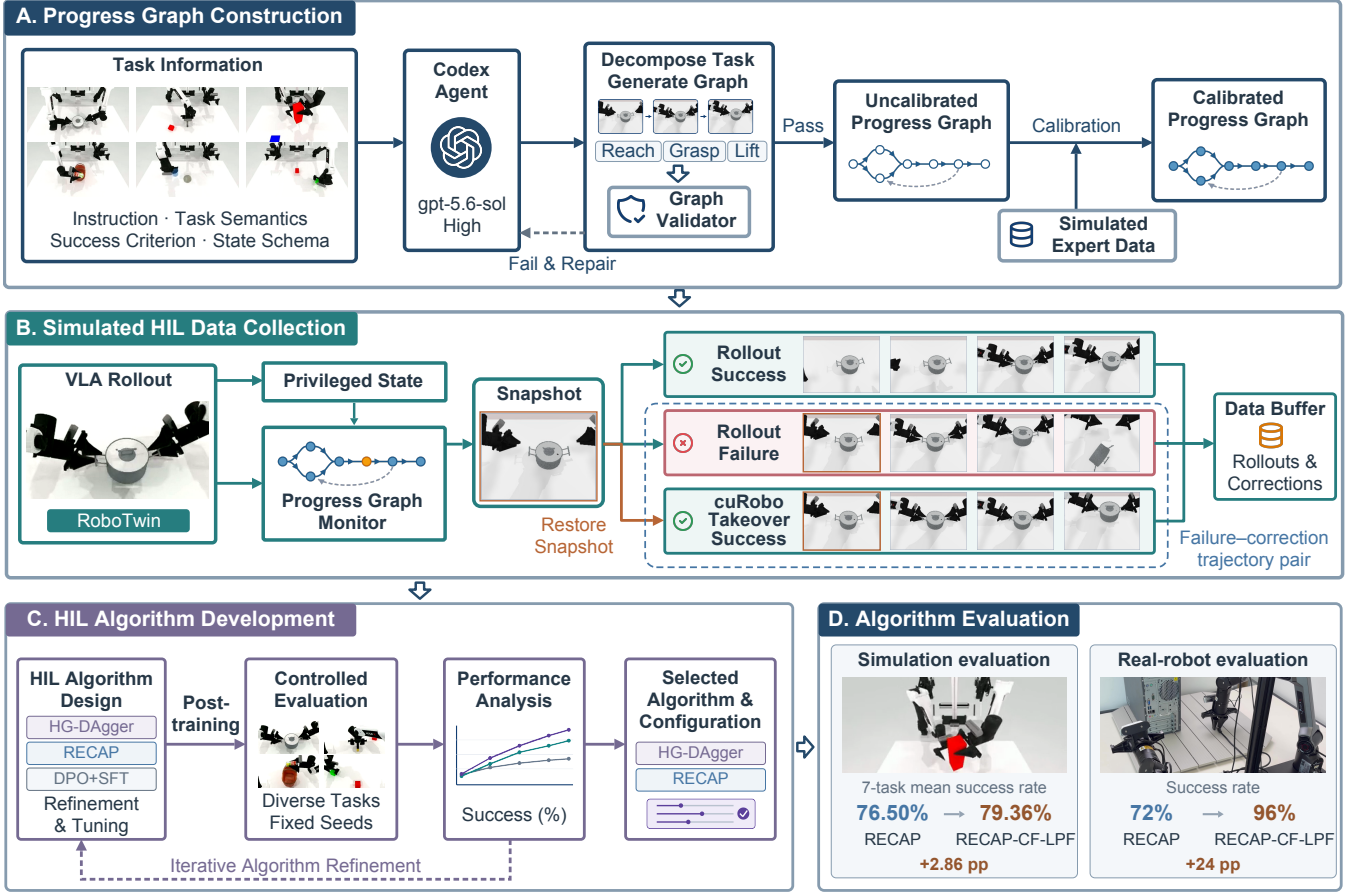


Fig. 1. SimPT integrates corrective data collection, VLA post-training, and standalone policy evaluation in an automated simulation framework. (A) An agent constructs and validates a Progress Graph, whose data-dependent thresholds are calibrated using simulated expert trajectories. (B) After terminal rollout failure, stable snapshots enable expert recovery and corrective data collection. (C) Post-training and controlled evaluation support algorithm comparison and refinement; curves are schematic. (D) RECAP and RECAP-CF-LPF are compared using seven-task mean success rates on held-out simulation trials and Ethernet cable-insertion success rates on a real robot. Gains are reported in percentage points (pp).

- We develop the Agent-Compiled Progress Graph to track task progress and retain stable snapshots during policy execution. After rollout failure, expert recovery starts from more advanced task stages and falls back to earlier ones as needed.
- We compare four post-training methods on seven RoboTwin tasks and use SimPT to develop RECAP-CF-LPF. By combining cross-fitting and low-pass filtering, this refinement improves policy success in both simulation and real-robot HIL post-training.

II. RELATED WORK

A. Intervention-Based Robot Learning

Interactive imitation learning mitigates distribution shift by collecting expert supervision during policy execution [18]. Dagger aggregates expert labels on states encountered by the learner and retrain the policy on the expanded dataset [4]. HIL variants differ in how they allocate expert control: HG-Dagger lets a human expert decide when to take over and return control [6], whereas ThriftyDagger requests assistance based on novelty and estimated risk within an intervention budget [19]. Their behavior-cloning policy updates use expert-

labeled data [20], without directly using the success or failure of autonomous rollouts as learning signals.

To learn from autonomous experience as well as expert corrections, recent VLA post-training methods incorporate reinforcement learning (RL) [21]. RECAP learns a value function to estimate future returns, accounting for remaining task steps and failure penalties, and uses the resulting advantages to condition policy training [15]. Noisy frame-level value estimates can nevertheless yield misleading advantages in OOD states. STEAM addresses this challenge by learning temporal-offset predictors from frame pairs in expert demonstrations and conservatively combining their advantage estimates [22]. Its ensemble reduces advantage overestimation at the cost of training and evaluating multiple predictors.

Preference optimization offers an alternative without a separately trained value function or critic. FlowPRO adapts direct preference optimization (DPO) [16] to flow-matching VLAs and uses a proximal regularizer to constrain implicit reward magnitudes and mitigate reward hacking. Its preference pairs are constructed from failed and corrective trajectories collected through human intervention and rollback [17]. Across these approaches, corrective data collection and

repeated real-robot evaluation demand substantial expert time and hardware resources, constraining systematic algorithm exploration.

B. Task Monitoring and Failure Recovery

Task monitoring connects execution feedback to failure detection and recovery [23], [24]. DoReMi uses VLM queries to detect task-constraint violations and trigger replanning [13]. Language-model agents also encode task knowledge in executable programs for robot planning and control, as demonstrated by Code as Policies [25], ProgPrompt [26], and VoxPoser [27]. Code-as-Monitor extends this idea to execution monitoring by compiling task constraints into code that checks tracked geometric elements, reducing reliance on repeated VLM queries [28]. AgentChord complements monitoring with context-aware recovery branches added to task graphs before execution, allowing detected failures to trigger prepared corrective behaviors [29]. We instead use an Agent-Compiled Progress Graph to guide expert recovery from stable snapshots after terminal rollout failure, generating corrective trajectories for VLA post-training.

C. Simulation for Data Generation and Policy Evaluation

Simulation supports both scalable training-data generation [30], [31] and controlled policy evaluation [32], [33]. RoboTwin combines generative digital twins with task code to generate demonstrations and benchmark manipulation policies [11]. Related benchmarks study knowledge transfer in lifelong robot learning, as in LIBERO [34], and unify policy evaluation across simulation and the real world, as in RoboDojo [35].

IntervenGen extends data generation to recovery behavior, focusing on robustness to object pose estimation errors. It combines closed-loop policy execution with transformed replay of human source interventions to synthesize corrective trajectories [12]. RLinf provides scalable infrastructure for embodied RL [36], while its real-world extension, USER, supports asynchronous online policy learning. Its HG-DAGger workflow relies on human experts to determine intervention timing and provide corrective actions [37]. Our simulation framework supports intervention-based VLA post-training without human takeover.

III. METHOD

A. Framework Overview

As shown in Fig. 1, SimPT uses a Progress Graph to connect policy execution with automated corrective data collection, followed by policy post-training and standalone evaluation. Language-model agents compile task specifications into candidate graphs, which undergo validation and selection. Scripts then calibrate the selected graph’s data-dependent thresholds using simulated expert trajectories. During policy execution, the graph tracks task progress and retains stable snapshots. If the rollout ends in failure, a programmatic expert attempts recovery from these snapshots to generate corrective trajectories. The collected rollouts and corrections support post-training under different learning objectives, while standalone evaluation compares the resulting

policies and guides algorithm development. After validating selected refinements in simulation, we apply them to real-robot HIL post-training, aiming to reduce the expert time and hardware resources needed for algorithm development.

B. Agent-Compiled Progress Graph

The Agent-Compiled Progress Graph (PG) consists of a task-stage graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and transition conditions evaluated on privileged simulator state s_t . Here, $\mathcal{V}$ is the set of task-stage nodes, and $\mathcal{E}$ is the set of directed edges for progression and regression. Forward edges activate subsequent stages when completion conditions are satisfied, while backward edges reactivate earlier stages when regression conditions are triggered.

To construct the graph, a preparatory agent reads the RoboTwin task source code and produces a structured specification containing the task instruction, object and arm assignments, available state fields, stage dependencies, and success criterion. Task-specific simulator adapters expose state fields such as object poses, gripper states, and contact measurements.

We pass this specification and a reusable library of state checks to K independent Codex agents based on GPT-5.6-Sol (Fig. 1A). Each agent compiles a candidate graph by assigning checks to stages and defining transitions for stage completion and loss of prerequisites. Candidate graphs undergo static validation against the specification, and the agents automatically revise them in response to validation feedback. Among valid candidates, we prefer fewer state fields, then fewer stages, and finally fewer checks. The selected graph specifies the task structure and symbolic checks, leaving data-dependent thresholds for subsequent calibration.

Fig. 2 illustrates the Progress Graph for Lift Pot. Before bimanual grasping, each arm must approach its assigned contact point on the pot. Separate branches track the left and right approach stages. Let $d_L(t)$ and $d_R(t)$ denote the distances from the respective tool center points to these contact points. The left and right approach stages are complete when

$$d_L(t) < \theta_{\text{app},L}, \quad d_R(t) < \theta_{\text{app},R}, \quad (1)$$

respectively, where $\theta_{\text{app},a}$ is the approach-distance threshold for arm $a \in \{L, R\}$. The branches merge at a join that verifies completion of both approach stages and checks that both arms satisfy the proximity conditions for bimanual grasping. Grasp loss during grasping or lifting can trigger regression to earlier stages. During a coordinated lift, for example, the right gripper may separate from the pot while the left gripper maintains its grasp. The graph detects this loss using the geometric condition $d_R(t) > \theta_{\text{loss}}$, where θ_{loss} is a fixed prior specified in the graph. It then reactivates the right approach stage and marks the dependent bimanual join, grasp, and lift stages as incomplete, while preserving completion of the left approach stage.

Before data collection, scripts calibrate data-dependent thresholds, such as $\theta_{\text{app},L}$ and $\theta_{\text{app},R}$, using simulated

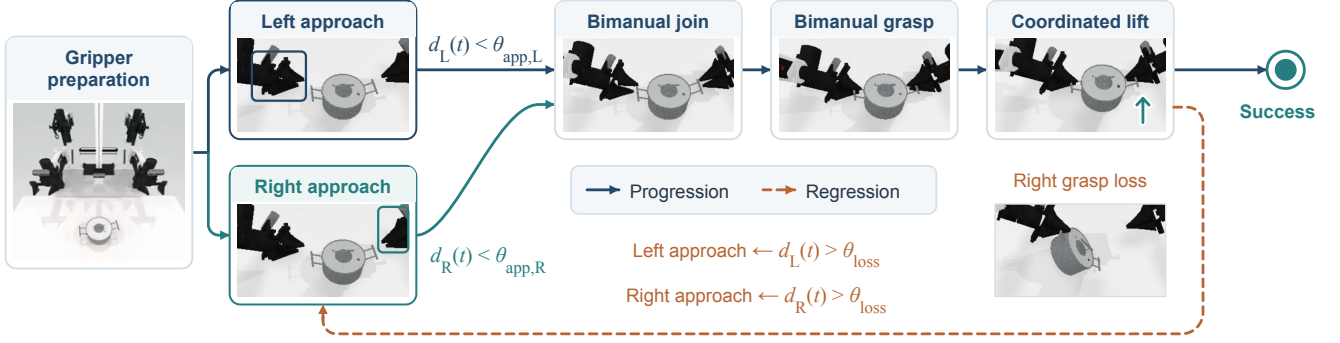


Fig. 2. Selected components of the Lift Pot Progress Graph illustrate stage completion and regression after grasp loss. Solid arrows indicate progression, while dashed orange arrows indicate regression. The approach, join, grasp, and lift images show successive states from one rollout; the grasp-loss image comes from a separate failed rollout. Gripper preparation is shown from an external viewpoint to display both arms.

expert trajectories. For the approach stages in Fig. 2, each threshold is estimated from an upper quantile of the distances measured when the corresponding arm completes its approach. The grasp-loss threshold θ_{loss} remains fixed at its graph prior, while task-success constants follow the original task definition. Using expert replays, the agent diagnoses and revises stage-completion and regression conditions. The graph is then recalibrated and validated through replay. The final graph and its parameters remain fixed during data collection. At runtime, state checks track task progress and guide recovery snapshot selection without further agent queries.

C. Snapshots and Expert Recovery

During policy execution, SimPT retains the latest snapshot recorded at each task stage. Each snapshot stores the simulator state, active stages, and capture step. We denote the set of retained snapshots at policy step t by

$$\mathcal{B}_t = \{\sigma_{init}, \sigma_1, \dots, \sigma_{N_t}\}, \quad (2)$$

where σ_{init} is the initial-state snapshot, σ_i denotes a snapshot retained during policy execution, and N_t is the number of retained non-initial snapshots.

When a rollout ends in failure, SimPT attempts expert recovery from the retained snapshots, starting with the most advanced task stages. For each attempt, it restores the saved simulator state and invokes a CuRobo-based programmatic expert to complete the remaining task. If recovery succeeds, the resulting trajectory is retained as corrective data. Otherwise, SimPT tries a snapshot from an earlier task stage. The initial-state snapshot serves as the final fallback, allowing the expert to restart the task.

D. Post-Training and Policy Evaluation

SimPT records successful and failed policy rollouts, together with expert corrective data, in a buffer for VLA post-training:

$$\mathcal{D} = \mathcal{D}_{succ}^{\pi} \cup \mathcal{D}_{fail}^{\pi} \cup \mathcal{D}_{corr}, \quad (3)$$

where $\mathcal{D}_{succ}^{\pi}$ and $\mathcal{D}_{fail}^{\pi}$ contain successful and failed rollouts of the VLA policy π , respectively, and $\mathcal{D}_{corr}$ contains expert corrective data. Each method selects data from this buffer and constructs training samples according to its learning objective.

For methods that require complete trajectories, SimPT constructs a corrected trajectory τ^+ from a failed rollout τ^- by appending the expert recovery segment to the policy prefix before recovery step t :

$$\begin{aligned} \tau^- &= \tau_{<t}^- \oplus \tau_{\geq t}^-, \\ \tau^+ &= \tau_{<t}^- \oplus \tau_t^E. \end{aligned} \quad (4)$$

Here, $\oplus$ denotes concatenation, and $\tau_{<t}^-$ is the policy prefix shared by the failed rollout and the complete corrected trajectory. The suffixes $\tau_{\geq t}^-$ and τ_t^E represent the failed policy continuation and successful expert recovery from the restored state, respectively. After post-training, standalone evaluation measures the updated policy's performance.

IV. EXPERIMENTS

A. Experimental Setup

To evaluate SimPT, we first compare PG with the fixed-interval Naive baseline for corrective data collection. We then implement and evaluate RECAP [15], HG-Dagger, DPO+SFT, and FlowPRO on seven RoboTwin manipulation tasks (Table I) [11]: Lift Pot, Move Can Pot, Place Empty Cup, Beat Block Hammer, Place Can Basket, Handover Block, and Stack Blocks Two. These tasks cover bimanual coordination, precise placement, tool use, and multistage manipulation. DPO+SFT and FlowPRO form preference pairs from failed and corrective trajectories. Following Appendix J of FlowPRO [17], DPO+SFT combines a flow-matching DPO loss with a supervised fine-tuning (SFT) objective, while FlowPRO adds a proximal regularizer. Finally, we use SimPT to refine RECAP in simulation and validate the refinement on a real-robot cable-insertion task.

For each task, Codex (GPT-5.6-Sol) generates three candidate graphs and performs up to two revisions per candidate using static-validation feedback. All policies use the $\pi_{0.5}$ backbone and an action chunk size of 50. We train a parent policy for each task using SFT on 50 expert demonstrations. Naive (N) and Progress Graph (PG) collection use the same parent policy checkpoint, 240 prescribed environment seeds, and CuRobo-based expert, with rollouts executed independently for each procedure. The Naive baseline saves snapshots every 50 policy steps and attempts recovery in reverse chronological order after task failure.

For each task and collection procedure, the methods share a fixed dataset collected with the SFT parent policy and construct training samples according to their learning objectives. All methods use 10,000 policy training steps. For each task, we use 40 validation seeds to tune parameters such as the RECAP guidance strength. Final held-out evaluation uses 200 fixed environment seeds per task that pass the official expert’s planning and task-success checks. These seeds are disjoint from the collection and validation sets, giving 1,400 trials per method across seven tasks. Standalone evaluation runs only the updated VLA, without the PG, recovery expert, or privileged simulator state.

B. Progress Graph Construction and Recovery

The agent produces candidate graphs that pass static validation for all seven tasks, with a final pass rate of 100% (21/21). Nine candidates (42.9%) pass on the first attempt. The remaining 12 require an average of 1.17 revisions, which the agent performs automatically using validation feedback. Graph generation and static validation take an average of 8.0 minutes per task, ranging from 4.5 to 11.5 minutes.

We compare PG and Naive in terms of search efficiency and recovery success. Each snapshot tested by the expert counts as one recovery attempt. We measure search efficiency by the total number of attempts and the mean attempts per successful recovery, computed as total attempts divided by recovered rollouts. Recovery success rate is the proportion of failed policy rollouts that the expert successfully recovers.

Table I shows that PG requires fewer recovery attempts than Naive on all seven tasks while maintaining or improving recovery success rates. Across tasks, PG reduces the total attempt count from 3,781 to 945, a reduction of 75.0%. The mean number of attempts per successful recovery falls from 7.62 to 1.79, while the overall recovery success rate rises from 93.4% to 98.9%. Handover Block shows the largest reduction in recovery attempts, from 1,329 to 192, together with an increase in recovery success rate from 83.6% to 98.3%. These results show more efficient recovery search under PG collection. We next compare the policies trained on the resulting datasets.

C. Comparison of Post-Training Methods

We first compare the four post-training methods using PG-collected data, with SFT as the baseline (Fig. 3). HG-Dagger achieves the highest overall success rate of 82.14% and performs best on five of the seven tasks. Its behavior-cloning objective directly supervises the policy with expert corrective actions. RECAP achieves 76.50%, exceeding the SFT baseline of 67.29%. Its advantage-conditioned training relies on value estimation [15]. With limited trajectory data, the value function can overfit, and temporal fluctuations in its predictions can compromise the reliability of advantage labels.

FlowPRO and DPO+SFT achieve overall success rates of 71.86% and 63.14%, respectively, both below RECAP; DPO+SFT also falls below SFT. Nevertheless, they perform

best on Beat Block Hammer and Place Empty Cup, respectively. Relative to DPO+SFT, FlowPRO adds a proximal regularizer that constrains the implicit reward magnitudes of preferred and rejected actions to mitigate reward hacking and stabilize preference optimization [17]. Both methods synthesize preferred action chunks by interpolating toward successful recovery trajectories. This interpolation does not account for object configurations or contact states, so action feasibility at failure states is not guaranteed.

We next compare each method using data from Naive and PG collection (Table II). PG collection improves the overall success rates of RECAP, HG-Dagger, and DPO+SFT, with the largest gain for RECAP. RECAP achieves 66.64% under Naive collection, slightly below SFT, and 76.50% under PG collection, a gain of 9.86 percentage points (pp), with improvements on all seven tasks. HG-Dagger and DPO+SFT gain 1.93 and 0.50 pp, respectively. FlowPRO is the exception: its success rate decreases from 76.07% under Naive collection to 71.86% under PG collection, a drop of 4.21 pp. Overall, PG-collected data improve policy success rates for most of the evaluated post-training methods.

D. Algorithm Exploration and Real-Robot Validation

Using SimPT, we develop *RECAP-CF-LPF* to refine the value-based advantage labels used by RECAP. Standard RECAP fits a value model to the collected trajectories and constructs advantage labels from predictions on those same trajectories, making the labels susceptible to value-model overfitting. We mitigate this issue with five-fold cross-fitting (CF), predicting values for each trajectory with a model trained on the other folds [38]. To attenuate high-frequency noise before computing 50-step advantages, we apply a second-order, zero-phase Butterworth low-pass filter (LPF) [39] with a 1 Hz cutoff to the out-of-fold value sequence sampled at 30 Hz. We compare this combined refinement with standard RECAP using PG-collected data from the seven tasks, then apply it to real-robot HIL post-training.

Fig. 4 compares value sequences during grasping in one Place Can Basket recovery trajectory. The standard RECAP prediction rises before gripper closure, whereas the CF prediction rises mainly during closure. In this example, the CF prediction is intuitively more plausible because its rise aligns more closely with the grasping motion. Applying LPF to the CF sequence attenuates local fluctuations before advantage computation.

Fig. 5 shows that RECAP-CF-LPF increases the overall validation success rate from 81.43% to 85.00%, a gain of 3.57 pp. Success rates improve on six tasks and remain unchanged on Place Empty Cup. Place Can Basket shows the largest gain, from 60.0% to 70.0%.

On the held-out evaluation set, RECAP-CF-LPF achieves 79.36% overall success, compared with 76.50% for standard RECAP, a gain of 2.86 pp across 1,400 trials per variant. Success rates increase on five tasks and decrease by 1.0 pp on Beat Block Hammer and 2.0 pp on Handover Block. The

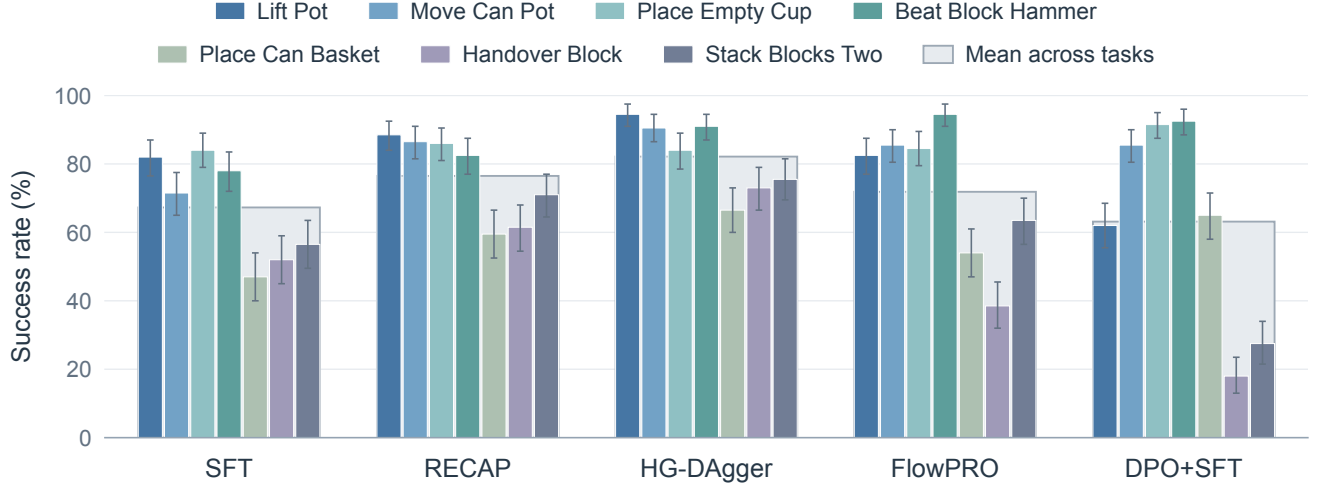


Fig. 3. Four post-training methods are compared with the SFT baseline on seven RoboTwin tasks. All post-training methods use data from Progress Graph (PG) collection. Standalone policy evaluation uses 200 held-out seeds per task, totaling 1,400 trials per method. Methods form groups on the horizontal axis, with seven colored bars per group corresponding to the tasks in legend order. Bar heights show task success rates; wide light-gray background bars span each group and show equal-weight means across tasks. Vertical whiskers indicate 95% bootstrap confidence intervals over evaluation seeds (20,000 resamples), conditional on the trained policies.

TABLE I

Recovery success and search efficiency of independent fixed-interval Naive (N) and Progress Graph (PG) collections across seven RoboTwin tasks. Attempts per recovery equal total attempts divided by recovered rollouts. Reduction columns report the relative decrease from N to PG using unrounded values; aggregate metrics use pooled counts. Bold indicates the better or tied N/PG result.

Task	Total attempts ↓			Attempts per recovery ↓			Recovery success (%) ↑	
	N	PG	Reduction (%)	N	PG	Reduction (%)	N	PG
Lift Pot	238	63	73.5	4.58	1.21	73.5	96.3	96.3
Move Can Pot	180	87	51.7	3.46	1.55	55.1	94.5	96.6
Place Empty Cup	89	46	48.3	2.70	1.12	58.4	100.0	100.0
Beat Block Hammer	76	66	13.2	1.58	1.40	11.3	100.0	100.0
Place Can Basket	1,295	345	73.4	10.88	2.72	75.0	93.7	100.0
Handover Block	1,329	192	85.6	11.87	1.63	86.3	83.6	98.3
Stack Blocks Two	574	146	74.6	7.17	1.68	76.6	100.0	100.0
Aggregate	3,781	945	75.0	7.62	1.79	76.5	93.4	98.9

TABLE II

Overall policy success rates are compared after post-training with data from Naive or Progress Graph (PG) collection. Each method is evaluated on 1,400 held-out trials per collection procedure across seven RoboTwin tasks. Δ denotes PG minus Naive in percentage points. Bold indicates the higher success rate within each row.

Method	Naive (%)	PG (%)	Δ (pp)
RECAP	66.64	76.50	+9.86
HG-DAGger	80.21	82.14	+1.93
DPO+SFT	62.64	63.14	+0.50
FlowPRO	76.07	71.86	-4.21

overall improvement has a 95% confidence interval of 0.36–5.36 pp, estimated using 20,000 paired bootstrap resamples of evaluation seeds within each task [40].

To test whether the refinement developed in simulation improves real-robot HIL post-training, we evaluate RECAP-CF-LPF on an Ethernet cable-insertion task. The platform comprises two AgileX PiPER arms with grippers and three RGB cameras: one head camera and one wrist camera per arm. Human experts operate two corresponding master arms

to collect demonstrations and corrections. Both variants use 1,672 training trajectories: 200 human demonstrations, 802 episodes with human interventions, and 670 fully autonomous policy rollouts.

As illustrated in Fig. 6, the robot grasps the cable, transfers it between arms, and aligns and inserts the plug into a desktop computer’s Ethernet port. The progress score starts at zero and increases by one after each milestone: cable grasping, bimanual handover, and plug insertion. Full insertion defines task success and yields the maximum score of three.

We evaluate 25 distinct scenarios varying cable length, initial position, and computer chassis placement. RECAP-CF-LPF succeeds in 24/25 scenarios (96%), compared with 18/25 (72%) for standard RECAP. All 18 scenarios solved by standard RECAP are also solved by RECAP-CF-LPF. The corresponding cumulative progress scores are 72/75 and 59/75, respectively, out of a maximum of 75 points across the 25 scenarios. These results show that the refinement

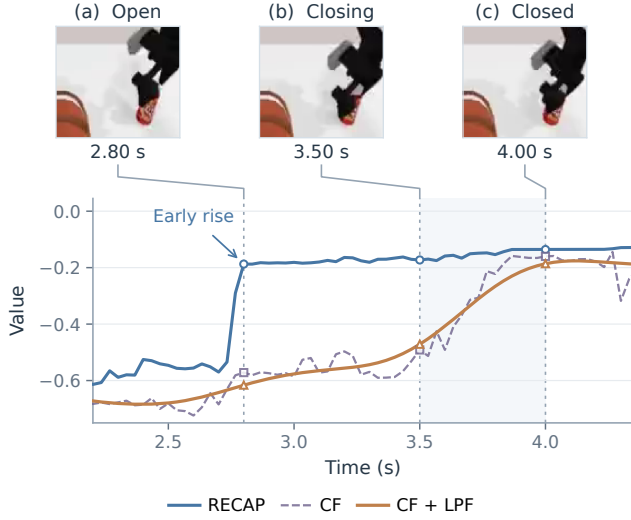


Fig. 4. Value sequences during grasping in a Place Can Basket expert recovery trajectory. Curves show standard RECAP, out-of-fold predictions from cross-fitting (CF), and the CF sequence after 1 Hz Butterworth low-pass filtering (RECAP-CF-LPF). Images (a–c) show the gripper open, closing, and closed, with dashed lines linking each image to its timestamp. The shaded interval spans the closing and closed gripper states (3.50–4.00 s).

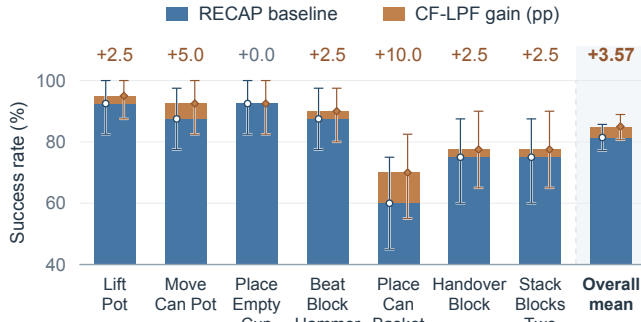


Fig. 5. Validation success rates on seven RoboTwin tasks using PG collection and 40 seeds per task. Blue bars show standard RECAP, and orange segments show the gain from RECAP-CF-LPF (five-fold cross-fitting and 1 Hz Butterworth low-pass filtering). Bar tops give RECAP-CF-LPF success rates; labels report gains in percentage points. Circles and diamonds mark standard RECAP and RECAP-CF-LPF estimates, respectively. Whiskers show 95% bootstrap confidence intervals over validation seeds for the selected policies (20,000 resamples). The vertical axis starts at 40%.

developed with SimPT improves policy performance when applied to real-robot HIL post-training.

V. CONCLUSION AND FUTURE WORK

We presented SimPT, a simulation framework for systematically exploring intervention-based VLA post-training algorithms. It automates policy rollouts, corrective data collection, policy training, and standalone evaluation. Its Agent-Compiled Progress Graph guides expert recovery after rollout failure, enabling corrective data collection without human takeover. Across seven RoboTwin tasks, PG achieved higher recovery success with fewer attempts than a fixed-interval baseline, and PG-collected data improved policy success for most evaluated methods. Using SimPT, we developed RECAP-CF-LPF, which combines cross-fitting and low-pass filtering to refine RECAP’s value-based advantage

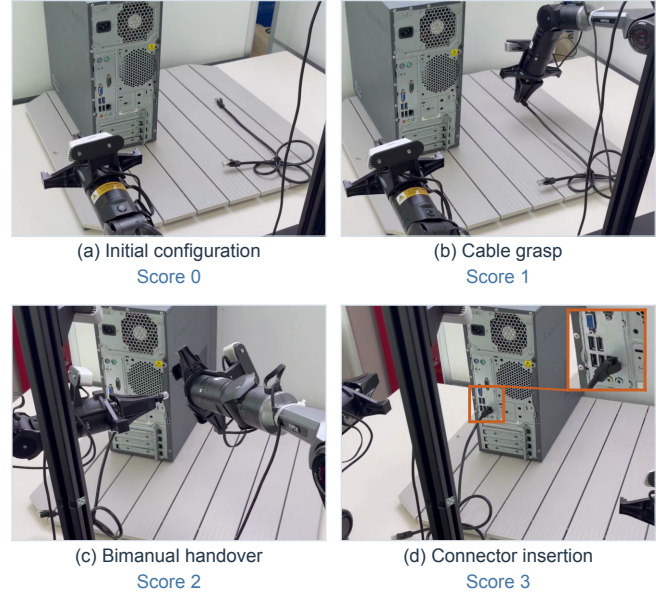


Fig. 6. Real-robot Ethernet cable insertion is evaluated using a cumulative progress score. Panels (a–d) show the initial configuration, cable grasping, bimanual handover, and plug insertion, corresponding to scores of 0, 1, 2, and 3. Each completed milestone adds one point, and full insertion defines task success. The inset in (d) provides a close-up of the inserted plug.

labels. The resulting gains in simulation and real-robot HIL post-training demonstrate SimPT’s utility for algorithm exploration and development.

Post-training performance remains sensitive to task and scene characteristics. We plan to incorporate real-to-sim reconstruction [41], [42] to better align simulated tasks and scenes with deployment conditions, strengthening SimPT as a platform for developing and evaluating real-robot HIL post-training methods.

REFERENCES

- [1] K. Black *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Proc. Robot. Sci. Syst. (RSS)*, Jun. 2025, doi: 10.15607/RSS.2025.XXI.010.
- [2] Physical Intelligence *et al.*, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” 2025, arXiv:2504.16054. [Online]. Available: <https://arxiv.org/abs/2504.16054>
- [3] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, “DART: Noise injection for robust imitation learning,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 78, 2017, pp. 143–156. [Online]. Available: <https://proceedings.mlr.press/v78/laskey17a.html>
- [4] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. Int. Conf. Artif. Intell. Stat. (AISTATS)*, ser. Proc. Mach. Learn. Res., vol. 15, 2011, pp. 627–635. [Online]. Available: <https://proceedings.mlr.press/v15/ross11a.html>
- [5] H. Ravichandar, A. S. Polydoros, S. Chernova, and A. Billard, “Recent advances in robot learning from demonstration,” *Annu. Rev. Control Robot. Auton. Syst.*, vol. 3, no. 1, pp. 297–330, 2020, doi: 10.1146/annurev-control-100819-063206.
- [6] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, “HG-Dagger: Interactive imitation learning with human experts,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2019, pp. 8077–8083, doi: 10.1109/ICRA.2019.8793698.
- [7] J. Luo, C. Xu, J. Wu, and S. Levine, “Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning,” *Sci. Robot.*, vol. 10, no. 105, Aug. 2025, Art. no. eads5033, doi: 10.1126/scirobotics.ads5033.

- [8] X. Li *et al.*, “Evaluating real-world robot manipulation policies in simulation,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 270, 2025, pp. 3705–3728. [Online]. Available: <https://proceedings.mlr.press/v270/li25c.html>
- [9] A. Mandlekar *et al.*, “MimicGen: A data generation system for scalable robot learning using human demonstrations,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 229, 2023, pp. 1820–1864. [Online]. Available: <https://proceedings.mlr.press/v229/mandlekar23a.html>
- [10] Z. Jiang *et al.*, “DexMimicGen: Automated data generation for bimanual dexterous manipulation via imitation learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, pp. 16 923–16 930, doi: 10.1109/ICRA55743.2025.11127809.
- [11] Y. Mu *et al.*, “RoboTwin: Dual-arm robot benchmark with generative digital twins,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 27 649–27 660, doi: 10.1109/CVPR52734.2025.02575.
- [12] R. Hoque, A. Mandlekar, C. Garrett, K. Goldberg, and D. Fox, “IntervenGen: Interventional data generation for robust and data-efficient robot imitation learning,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2024, pp. 2840–2846, doi: 10.1109/IROS58592.2024.10801523.
- [13] Y. Guo, Y.-J. Wang, L. Zha, and J. Chen, “DoReMi: Grounding language model by detecting and recovering from plan-execution misalignment,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2024, pp. 12 124–12 131, doi: 10.1109/IROS58592.2024.10802284.
- [14] B. Sundaralingam *et al.*, “CuRobo: Parallelized collision-free robot motion generation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 8112–8119, doi: 10.1109/ICRA48891.2023.10160765.
- [15] Physical Intelligence *et al.*, “ $\pi_{0.6}^*$: a VLA that learns from experience,” 2025, arXiv:2511.14759. [Online]. Available: <https://arxiv.org/abs/2511.14759>
- [16] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 53 728–53 741, doi: 10.52202/075280-2338. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/a85b405ed65c6477a4fe8302b5e06ce7-Paper-Conference.pdf
- [17] Y. Wu, H. Zhang, J. Tan, X. Wang, and Z. Zhang, “FlowPRO: Reward-free reinforced fine-tuning of flow-matching VLAs via proximalized preference optimization,” 2026, arXiv:2606.05468. [Online]. Available: <https://arxiv.org/abs/2606.05468>
- [18] J. Zhang and K. Cho, “Query-efficient imitation learning for end-to-end simulated driving,” *Proc. AAAI Conf. Artif. Intell.*, vol. 31, no. 1, pp. 2891–2897, 2017, doi: 10.1609/aaai.v31i1.10857.
- [19] R. Hoque, A. Balakrishna, E. Novoseller, A. Wilcox, D. S. Brown, and K. Goldberg, “ThriftyDagger: Budget-aware novelty and risk gating for interactive imitation learning,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 164, 2022, pp. 598–608. [Online]. Available: <https://proceedings.mlr.press/v164/hoque22a.html>
- [20] C. Celemin *et al.*, “Interactive imitation learning in robotics: A survey,” *Found. Trends Robot.*, vol. 10, no. 1–2, pp. 1–197, 2022, doi: 10.1561/23000000072.
- [21] H. Deng *et al.*, “A survey on reinforcement learning of vision-language-action models for robotic manipulation,” 2025, TechRxiv preprint, doi: 10.36227/techrxiv.176531955.54563920/v1.
- [22] Z. Liu *et al.*, “STEAM: Self-supervised temporal ensemble advantage modeling for real-world robot learning,” 2026, arXiv:2606.29834. [Online]. Available: <https://arxiv.org/abs/2606.29834>
- [23] W. Huang *et al.*, “Inner monologue: Embodied reasoning through planning with language models,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 205, 2023, pp. 1769–1782. [Online]. Available: <https://proceedings.mlr.press/v205/huang23c.html>
- [24] Z. Liu, A. Bahety, and S. Song, “REFLECT: Summarizing robot experiences for failure explanation and correction,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 229, 2023, pp. 3468–3484. [Online]. Available: <https://proceedings.mlr.press/v229/liu23g.html>
- [25] J. Liang *et al.*, “Code as Policies: Language model programs for embodied control,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 9493–9500, doi: 10.1109/ICRA48891.2023.10160591.
- [26] I. Singh *et al.*, “ProgPrompt: Generating situated robot task plans using large language models,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 11 523–11 530, doi: 10.1109/ICRA48891.2023.10161317.
- [27] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3D value maps for robotic manipulation with language models,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 229, 2023, pp. 540–562. [Online]. Available: <https://proceedings.mlr.press/v229/huang23b.html>
- [28] E. Zhou *et al.*, “Code-as-Monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 6919–6929.
- [29] S. Xu *et al.*, “From reaction to anticipation: Proactive failure recovery through agentic task graph for robotic manipulation,” in *Proc. Robot. Sci. Syst. (RSS)*, Sydney, Australia, Jul. 2026, doi: 10.15607/RSS.2026.XXII.180.
- [30] Y. Wang *et al.*, “RoboGen: Towards unleashing infinite data for automated robot learning via generative simulation,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 235, 2024, pp. 51 936–51 983. [Online]. Available: <https://proceedings.mlr.press/v235/wang24cc.html>
- [31] L. Wang *et al.*, “GenSim: Generating robotic simulation tasks via large language models,” in *Int. Conf. Learn. Represent. (ICLR)*, 2024. [Online]. Available: <https://gen-sim.github.io/>
- [32] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, “RLBench: The robot learning benchmark & learning environment,” *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 3019–3026, 2020, doi: 10.1109/lra.2020.2974707.
- [33] J. Gu *et al.*, “ManiSkill2: A unified benchmark for generalizable manipulation skills,” in *Int. Conf. Learn. Represent. (ICLR)*, 2023. [Online]. Available: <https://maniskill2.github.io/>
- [34] B. Liu *et al.*, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 44 776–44 791, doi: 10.52202/075280-1939.
- [35] T. Chen *et al.*, “RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies,” 2026, arXiv:2607.04434. [Online]. Available: <https://arxiv.org/abs/2607.04434>
- [36] C. Yu *et al.*, “RLinf: Flexible and efficient large-scale reinforcement learning via macro-to-micro flow transformation,” in *Proc. USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, Seattle, WA, Jul. 2026, pp. 829–846. [Online]. Available: <https://www.usenix.org/conference/osdi26/presentation/you-chao>
- [37] H. Zang *et al.*, “USER: A unified and extensible system for online real-world policy learning in embodied AI,” in *Proc. Robot. Sci. Syst. (RSS)*, Sydney, Australia, Jul. 2026, doi: 10.15607/RSS.2026.XXII.037.
- [38] V. Chernozhukov *et al.*, “Double/debiased machine learning for treatment and structural parameters,” *Econom. J.*, vol. 21, no. 1, pp. C1–C68, 2018, doi: 10.1111/ectj.12097.
- [39] S. Butterworth, “On the theory of filter amplifiers,” *Exp. Wireless Wireless Eng.*, vol. 7, pp. 536–541, Oct. 1930.
- [40] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.
- [41] M. Torne Villasevil *et al.*, “Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation,” in *Proc. Robot. Sci. Syst. (RSS)*, Delft, Netherlands, Jul. 2024, doi: 10.15607/RSS.2024.XX.015.
- [42] X. Han *et al.*, “RE³SIM: Generating high-fidelity simulation data via 3D-photorealistic real-to-sim for robotic manipulation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026. [Online]. Available: <https://re3sim.github.io/>